Static Reflection in a Nutshell

- Document number: **P0578R1**, ISO/IEC JTC1 SC22 WG21
- Date: 2017-06-18

- Authors: Matúš Chochlík chochlik@gmail.com, Axel Naumann axel@cern.ch, and David Sankel dsankel@bloomberg.net

- Audience: LEWG

Abstract

This paper is a concise introduction to the static reflection facilities proposed in P0194. See also P0385 for examples, in-depth design rational, and future extensions. A previous version of the functionality described here has been implemented in clang and is available for experimentation.

History

- 2017-06-13. **P0578R1**. Modified to reflect the outcome of the 2016 Oulu meeting where `reflexpr` was preferred to `$reflect`. The `get_base_name` operation was changed to `get_name` which only works for types that have a name (e.g., `const` and `&` qualified types do not have a name).
- 2016-02-04. **P0578R0**. The initial version of this paper.

Introduction

C++ provides many facilities that allow for the compile-time inspection and manipulation of types and values. Indeed an entire field, template metaprogramming, arose to take advantage of these features. While the feats accomplished have been both surprising and impressive, C++ still lacks some fundamental building blocks in this area. There is no way, for example, to discover an arbitrary class's name or query its member variables. This paper describes an attempt to extend the language with the most crucial of these building blocks.

Lets look at a simple example illustrating our proposed design:

```
template <typename T>
T min(const T& a, const T& b) {
  log() << "min<"
        << get_display_name_v<reflexpr(T)>
        << ">(" << a << ", " << b << ") = ";
  T result = a < b ? a : b;
  log() << result << std::endl;
  return result;
}
```

Here we define a `min` function that closely resembles, aside from the logging, the semantics of `std::min`. Assuming several calls, the output of this function might look something like this:

```
min<int>(1, 33)
min<std::string>(hello, world)
...
```

Note that the *type* argument as well as the value arguments are printed out. Our proposed reflection syntax and library is what makes this possible.

The key expression is `get_display_name_v<reflexpr(T)>`, which consists of two parts. The first is `reflexpr(T)`. This expression produces a special type that contains meta-information concerning `T` (e.g. name, member variable, and inheritance information). We call these types "meta objects" following industry practice. `reflexpr`'s argument doesn't necessarily have to be a type though; many kinds of syntax are recognized in the general case. The second part is the call to `get_display_name_v`. This is where we extract a piece of information from the meta object, in this case the "display name". Most uses of the reflection facilities we provide are variations of this simple theme: reflect to produce a metaobject and query that meta object for information.

Our Approach

The design space for reflection is vast. It is tempting to set complete coverage as a goal even though the value proposition is dubious at best. Instead, we attempted a pragmatic approach where we add a minimal set of features that are tied to concrete use cases and compliment the existing C++ feature set. You will not see, for instance, a replacement or alternative for `std::is_const` or other well-established pre-existing reflection capabilities.

We also are not, in this iteration, proposing a high-level reflection API. There is still a lot of discussion as to what such an API should look like. More experimentation is required. Instead we provide low-level interfaces that can be built upon.

Our vision for a complete reflection software stack is as follows

User Code
↑
Domain Specific Library
↑
Reflection Library
↑
Reflection Facilities

At the bottom are low-level reflection facilities. That is what we are proposing here. On top of that is a higher-level reflection library that, building on the lower-level facilities, provides an easy and convenient way to write libraries requiring reflection. Higher yet are domain specific libraries. This could be a serialization library or an automatic database schema generator. Finally, on the top, is user code which makes use of the domain specific libraries.

The danger of designing facilities instead of a high-level API is that the former ends up inadequate for the latter. Fortunately, we've been developing in parallel several variations of high-level APIs that make use of our facilities. See the following snippet making use of the mirror library, a Boost.MPL style reflection library built on our facilities:

```
template <typename T>
bool generic_equal(const T& a, const T& b)
{
  using metaT = reflexpr(T);
  bool result = true;
  mirror::for_each<mirror::get_data_members_t<metaT>>(
    compare_data_members<T>{a, b, result}
  );
  return result;
}
```

At the 2016 Issaquah meeting, Louis Dionne presented a Boost.Hana-styled reflection library built on the facilities described here along with an impressive JSON serialization library built on top of that. We find this result encouraging and suggestive that our facilities are as general and API-agnostic as we had hoped.

What's In and What's Out

As mentioned above, we're aiming for a minimal set of functionality that still satisfies a number of use cases. Here's a basic summary of what's included and what's not.

Included:

- **Data members**. e.g. walking through the data members of a class
- **Member types**. e.g. walking through nested types or `typedef`s in a class
- **Enumerators**. The ability to, for example, make one-line serialization routines for `enum`s
- **Template instantiations**. The ability to reflect on instantiated templates, such as `std::vector<int>`
- **Alias support**. The ability to distinguish between a `typedef` and its underlying type

Not Included:

- **Namespace member sequences**. We're not providing the ability to walk through all the declarations of a namespace. Efficient implementations of such a feature seem unlikely and use cases are wanting.
- **Functions**. We think we know what we want and how we can do it, but this chapter is complex. This is comming in a separate proposal, P0670.
- **Class templates**. It is unclear, as of yet, what reflecting on an uninstantiated template, like `vector`, would look like.
- **Building new datatypes**. We have plans to extend this proposal at some point with the ability to build new datatypes with an identifier generation facility. See the design document for more details.
- **Reflection facilities already in C++**. As mentioned before, we're in the business of cooperating with existing facilities

instead of replacing what already exists.

- **Anonymous functions**. An extension to this paper which supports anonymous functions will be forthcoming.
- **Attributes**. We feel that reflecting on attributes would be a highly valuable extension to this proposal. However, this feature would cause a significant change to how we use attributes today. We intend to propose this in a future paper.

Language Considerations

The primary consideration at the language level was what to call the `reflexpr` operator. There was a bikeshedding at the 2016 Kona meeting and `reflexpr` had the most consensus when compared to several other options.

Library Considerations

strings

The lack of a decent compile-time string representation forced us to choose between several undesirable options. For consistency with the other metafunctions we decided to use `integral_constant<const char (&)[N], STR>` where `STR` is the name of a static, null-terminated byte string of length `N`.

We assume WG21 will incorporate proper compile-time strings at some point and consider this a placeholder implementation.

`ObjectSequence`

Because there isn't a native type-list implementation in the standard library, we included a placeholder implementation in the reflection proposal. It provides the ability to query size and get an element by index.

```
template <ObjectSequence T>
  constexpr auto get_size_v = get_size<T>::value;

template <size_t I, ObjectSequence S>
  using get_element_t = typename get<I, S>::type;
```

Additionally, the `unpack_sequence_v` metafunction was provided that enables the convenient conversion of a `ObjectSequence` into another type-list representation, such as a `std::tuple`.

```
template <template <class...> class Tpl, ObjectSequence S>
  constexpr auto unpack_sequence_v = unpack_sequence<Tpl, S>::value;
```

We assume WG21 will incorporate proper compile-time type lists at some point and consider this a placeholder implementation as well.

Concepts

All types containing metainformation satisfy the `reflect::Object` concept. Beyond that, there are several other concepts that provide more specialized information. Generally a metaobject will satisfy several of the concepts below.

Class-like things:

- `Record`. A `union` or a `Class`
- `Class`. A `class` or a `struct`
- `Base`. The `public B` part of `class A : public B {};`, for example
- `RecordMember`. Data members and member types

Scopes:

- `Namespace`. A namespace
- `Scope`. A namespace, class, or enumeration scope
- `ScopeMember`. Something in a scope
- `GlobalScope`. The global namespace, `::`

Enums:

- `Enum`. An `enum`.
- `Enumerator`. An enumerator.

Types:

- `Typed`. Something with a type, such as a member variable.
- `Type`. A type.

Expressions:

- `Variable`. A variable.
- `Constant`. A constant expression, like an enumerator.

Other:

- `Named`. Something with a name.
- `Alias`. An alias, such as a `typedef`.
- `ObjectSequence`. Our stand-in for type lists.

In the following sections we'll go into more detail for some of the more important concepts and operations.

Object

The `reflexpr` operation always produces a type that satisfies the `Object` concept. `Object`s provide the ability to query source location and the `reflects_same` method determines whether or not two objects reflect the same underlying entity.

```
template <Object T> struct get_source_line;
template <Object T> struct get_source_column;
template <Object T> struct get_source_file_name;

template <Object T1, Object T2>
struct reflects_same;
```

Additionally, `get_source_location` is provided for compatibility with the `std::source_location` datatype in the library fundamentals TS.

```
template <Object T>
struct get_source_location;
// return a std::source_location object
```

Record

A record is a `union`, `class`, or `struct` (i.e. a class type).

The most general way to query members is through use of `get_data_members` and `get_member_types`. These provide lists of all the public, private, and protected members. Because the access of private members can cause abstraction leaks, two other variants are provided.

The `get_public_*` metafunctions return only the public members of a record. This operation can be used safely on third party code. The `get_accessible_*` metafunctions, on the other hand, will include private members as well if the `reflexpr` operation's surrounding context allows it (e.g. it is found in a member function or friend class). Encapsulation cannot be broken with either of these two variants.

```
template <Record T> struct get_data_members;
template <Record T> struct get_public_data_members;
template <Record T> struct get_accessible_data_members;

template <Record T> struct get_member_types;
template <Record T> struct get_public_member_types;
template <Record T> struct get_accessible_member_types;
```

Named

Most entities that can be reflected upon have a name of some sort and the `Named` concept supports this. There are two primary string-returning operations, `get_display_name` and `get_name`.

```
template <Named T> struct get_display_name;
template <Named T> struct get_name;
```

While the semantics of both of these functions is implementation defined, they have a clear difference in intent as described below.

`get_name`

`get_name` returns the name of the underlying reflected entity if it has one. Note that abbreviations are elaborated and instantiated template classes return the name of the template class itself. Decorated types, such as those with `const` and `volatile` do not have a name and the `get_name` operation on these returns "".

```
get_name_v<reflexpr(unsigned)>
// "unsigned int"

using foo = int;
get_name_v<reflexpr(foo)>
// "foo"

get_name_v<reflexpr(std::vector<int>)>
// "vector"

get_name_v<reflexpr(volatile std::size_t* [10])>
// ""
```

`get_display_name`

`get_display_name` provides a way for compilers to provide a non-portable, but human readable, representation of the underlying entity. The intent is for this to hook into the technology already used in compilers to provide human readable diagnostics.

```
get_display_name_v<reflexpr(unsigned)>
// "unsigned"

using foo = int;
get_display_name_v<reflexpr(foo)>
// "foo"

get_display_name_v<reflexpr(std::vector<int>)>
// "std::vector<int>"

get_display_name_v<reflexpr(volatile std::size_t* [10])>
// "volatile std::size_t *[10]"
```

`Alias`

Aliases (viz. `typedef`s, etc.) provide a `get_aliased` operation which returns the underlying entity being reflected.

```
template <Alias T> struct get_aliased;
```

For example:

```
using MyInt = int;

get_name_v<reflexpr(MyInt)> // "MyInt"
get_name_v<get_aliased_t<reflexpr(MyInt)>> // "int"
```

Note that `get_aliased_t` always returns the true underlying type. Walking through `typedef`s of `typedef`s is not supported as a concession to compiler implementers.

Conclusion

We've overviewed a proposal for adding static reflection to C++. The feature set provided here goes a long way towards filling the holes metaprogrammers face today and will be extended to support even more ambitious reflection capabilities in the future.